Programação Estruturada em C

# Aula 2 — Condicionais, expressões lógicas e switch-case

Prof. Me. Eng. Matheus Kotaki

# O que muda a partir de hoje?

### Programa sequencial

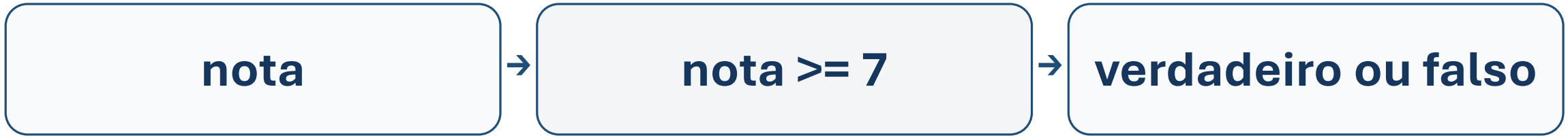
- Recebe dados
- Executa os comandos em sequência
- O caminho é sempre o mesmo

### Programa com decisão

- Recebe dados
- Avalia uma condição
- O caminho pode mudar conforme os valores

# Uma condição é uma pergunta sobre os dados

**Regra:** “O aluno está aprovado se a nota for pelo menos 7.”



# Operadores relacionais: compare valores

| Operador | Leitura |
|---|---|
| == | igual a |
| != | diferente de |
| > | maior que |
| < | menor que |
| >= | maior ou igual a |
| <= | menor ou igual a |

> “18 anos ou mais” → idade >= 18

> “valor diferente de zero” → valor != 0

# Verdadeiro e falso em C

## Expressão falsa

- Em C, zero é interpretado como falso
- Ex.: 5 > 10 produz falso

## Expressão verdadeira

- Valores diferentes de zero são interpretados como verdadeiros
- Ex.: 10 >= 10 produz verdadeiro

```
#include <stdio.h>

int main(){
	printf("%d", 5==5);
	printf("%d", 5==6);
	return 0;
}
```

# Primeira decisão com if

```
if (idade >= 18) {
    printf("Maior de
idade\n");
}
```

Se a condição for verdadeira, o bloco é executado.

Se for falsa, o bloco é ignorado e o programa continua depois dele.

**Com idade = 17, a mensagem aparece?**

# Rastreando um if

```
int idade = 22;

if (idade >= 18) {
    printf("Acesso permitido\n");
}

printf("Fim\n");
```

## Rastreamento

idade = 22

↓

22 >= 18

↓

verdadeiro

↓

executa o bloco

**Depois do if, a execução continua normalmente.**

# Quando existem dois caminhos: if-else

```
if (nota >= 7.0) {
    printf("Aprovado\n");
} else {
    printf("Reprovado\n");
}
```

**Uma condição, dois caminhos possíveis**

verdadeiro → executa o bloco do if

falso → executa o bloco do else

**Apenas um dos dois blocos será executado.**

**Para nota = 7.0, qual bloco será executado?**

# Atenção: == compara; = atribui

## Comparação

```
if (idade == 18) {
    printf("Idade igual a 18\n");
}
```

## Atribuição — erro lógico

```
if (idade = 18) {
    printf("Idade igual a 18\n");
}
```

# Condições compostas

# && — todas as condições precisam ser verdadeiras

**Regra: aprovação exige nota mínima e frequência mínima.**

```
if (nota >= 7 && frequencia >= 75) {
    printf("Aprovado\n");
}
```

&& significa: as duas partes devem ser verdadeiras.

**nota = 8 e frequência = 70: aprovado?**

# || — basta uma das condições ser verdadeira

**Regra: desconto para estudante ou pessoa com 60 anos ou mais.**

```
if (estudante == 1 || idade >= 60) {
    printf("Desconto aplicado\n");
}
```

|| significa: pelo menos uma parte precisa ser verdadeira.

**idade = 65 e estudante = 0: recebe desconto?**

# ! — nega uma condição

**Use quando a regra for naturalmente expressa como “não …”.**

```
if (!bloqueado) {
    printf("Acesso permitido\n");
}
```

> !bloqueado pode ser lido como “não está bloqueado”.

# Construa a condição a partir da frase

**“A nota deve estar entre 0 e 10, inclusive.”**

**nota >= 0** → **E** → **nota <= 10**

```
nota >= 0 && nota <= 10
```

**Por que usar || aqui deixaria a condição errada?**

# Parênteses ajudam a deixar a intenção clara

```
if ((idade >= 18 && possuiIngresso == 1) || autorizado == 1)
{
    printf("Entrada permitida\n");
}
```

# Prática 1 — leitura, previsão, modificação

```
int idade;
printf("Idade: ");
scanf("%d", &idade);

if (idade >= 18) {
    printf("Maior de idade\n");
} else {
    printf("Menor de idade\n");
}
```

- Preveja o resultado para 17, 18 e 19.
- Execute e compare com sua previsão.
- Modifique a regra para 16 anos ou mais.
- Teste novamente os valores próximos ao limite.

# Mais de dois resultados: else if

```
if (nota >= 7.0) {
    printf("Aprovado\n");
} else if (nota >= 5.0) {
    printf("Recuperacao\n");
} else {
    printf("Reprovado\n");
}
```

As condições são verificadas na ordem.

Assim que uma condição for verdadeira, aquele caminho é executado e os seguintes não são avaliados.

**Para nota = 6, qual caminho será executado?**

# A ordem das condições importa

```
if (nota >= 5) {
    printf("Recuperacao\n");
} else if (nota >= 7) {
    printf("Aprovado\n");
} else {
    printf("Reprovado\n");
}
```

**O programa consegue chegar ao segundo teste com nota 8?**

# Condição Aninhada

```
if (idade >= 18) {
    if (autorizacao == 1) {
        printf("Entrada permitida\n");
    } else {
        printf("Falta autorizacao\n");
    }
} else {
    printf("Idade insuficiente\n");
}
```

idade suficiente → verificar autorização

> A segunda condição só é testada se a primeira condição for verdadeira.

**Se idade = 16, a autorização é verificada?**

# Dois if independentes não são o mesmo que if-else

### Duas decisões independentes

```
if (x > 0) {
    printf("Positivo\n");
}

if (x % 2 == 0) {
    printf("Par\n");
}
```

### Dois caminhos exclusivos

```
if (x > 0) {
    printf("Positivo\n");
} else {
    printf("Nao positivo\n");
}
```

# switch-case

# Um menu usando else if

```
if (opcao == 1) {
    printf("Consultar saldo\n");
} else if (opcao == 2) {
    printf("Depositar\n");
} else if (opcao == 3) {
    printf("Sacar\n");
} else if (opcao == 4) {
    printf("Sair\n");
} else {
    printf("Opcao invalida\n");
}
```

Sempre comparamos a mesma variável com valores específicos.

# O mesmo menu com switch-case

```
switch (opcao) {
    case 1:
        printf("Consultar saldo\n");
        break;
    case 2:
        printf("Depositar\n");
        break;
    case 3:
        printf("Sacar\n");
        break;
    case 4:
        printf("Sair\n");
        break;
    default:
        printf("Opcao invalida\n");
}
```

case: valor que queremos tratar

break: encerra aquele caminho do switch

default: caminho para valores não contemplados

# O que acontece sem break?

```
int opcao = 1;

switch (opcao) {
    case 1:
        printf("Um\n");
    case 2:
        printf("Dois\n");
        break;
}
```

**Com opcao = 1, quais mensagens serão exibidas?**

# Prática 2 — escolha a estrutura

## Problema A

Classifique uma temperatura como baixa, confortável ou alta, usando limites definidos no enunciado.

## Problema B

Receba um código: 1 = financeiro, 2 = suporte, 3 = comercial, 4 = cancelamento. Trate código inválido.

# Prática 2 — Problema A

Crie um programa que leia uma temperatura e a classifique em uma das três categorias:

**Baixa:** temperatura menor que 18 °C;

**Confortável:** temperatura entre 18 °C e 26 °C, inclusive;

**Alta:** temperatura maior que 26 °C.

O programa deve exibir a classificação correspondente.

Teste o programa com valores que representem cada faixa e também com valores próximos aos limites, como 17, 18, 26 e 27.

# Prática 2 — Problema B

Crie um programa que leia um código inteiro e informe o setor correspondente:

1 — Financeiro;

2 — Suporte;

3 — Comercial;

4 — Cancelamento.

Caso seja informado qualquer outro código, o programa deve exibir a mensagem **"Código inválido"**.